

SESSION 7

รูปแบบขั้นสูง

Optimistic Updates · Polling · Pagination · Custom Middleware

หลักสูตร React-Redux · โปรเจกต์ AcadeMate

Dr.Watcharin Sarachai

Optimistic Updates

UI ทันที, ย้อนกลับอัตโนมัติ

Polling & Refetch

ข้อมูลสด, ซิงค์อัปเดตระยะ

การแบ่งหน้า

รูปแบบ Infinite Scroll

Middleware

ปลั๊กอิน Store แบบกำหนดเอง

Session 7 — เนื้อหาและเป้าหมายการเรียนรู้

ส่วนที่ 1 — Optimistic Updates

ปรับ UI ก่อนเซิร์ฟเวอร์ตอบสนอง ย้อนกลับอัตโนมัติเมื่อล้มเหลวด้วย `onQueryStarted` + `util.updateQueryData`

ส่วนที่ 2 — Polling & Refetching

รักษาข้อมูลให้ทันสมัยด้วย `pollingInterval`, `refetchOnFocus`, `refetchOnReconnect` และ `refetch` แบบ manual

ส่วนที่ 3 — การแบ่งหน้า

รูปแบบแบ่งหน้าและ infinite scroll โดยใช้ `serializeQueryArgs`, `merge` และ `forceRefetch`

ส่วนที่ 4 — Custom Middleware

ดักจับทุก `dispatch` เพื่อเพิ่ม logging, error toasts และ analytics ใน Redux store

Lab 7 — AcadeMate ขั้นสูง

เพิ่ม optimistic updates ให้ `updateStudent`, เปิดใช้ live polling และเชื่อมต่อ logger middleware

Optimistic Updates

อัปเดต UI ทันที — ก่อนที่เซิร์ฟเวอร์จะตอบสนอง ย้อนกลับอัตโนมัติหากล้มเหลว

Optimistic Updates คืออะไร?

อัปเดต UI ทันที — ก่อนเซิร์ฟเวอร์ยืนยัน ย้อนกลับอัตโนมัติหากคำขอล้มเหลว

ไม่มี Optimistic Update

1 ผู้ใช้คลิก 'บันทึก'

UI รอ — ปุ่มถูกปิดใช้งาน แสดง spinner

2 ส่งคำขอ → รอ...

ล่าช้า 300ms–2s บนเครือข่ายช้า

3 เซิร์ฟเวอร์ยืนยัน → UI อัปเดต

ตารางแสดงข้อมูลใหม่ในที่สุด

4 เกิดข้อผิดพลาด → แสดงข้อความ

ผู้ใช้ต้องลองใหม่ — UX แย่

มี Optimistic Update

1 ผู้ใช้คลิก 'บันทึก'

UI อัปเดตทันที — ไม่ต้องรอ

2 ส่งคำขอในเบื้องหลัง

ผู้ใช้ทำงานต่อได้ทันที

3 เซิร์ฟเวอร์ยืนยัน → คงการแก้ไข

แคชถูกต้องแล้ว — ไม่ต้องทำอะไร

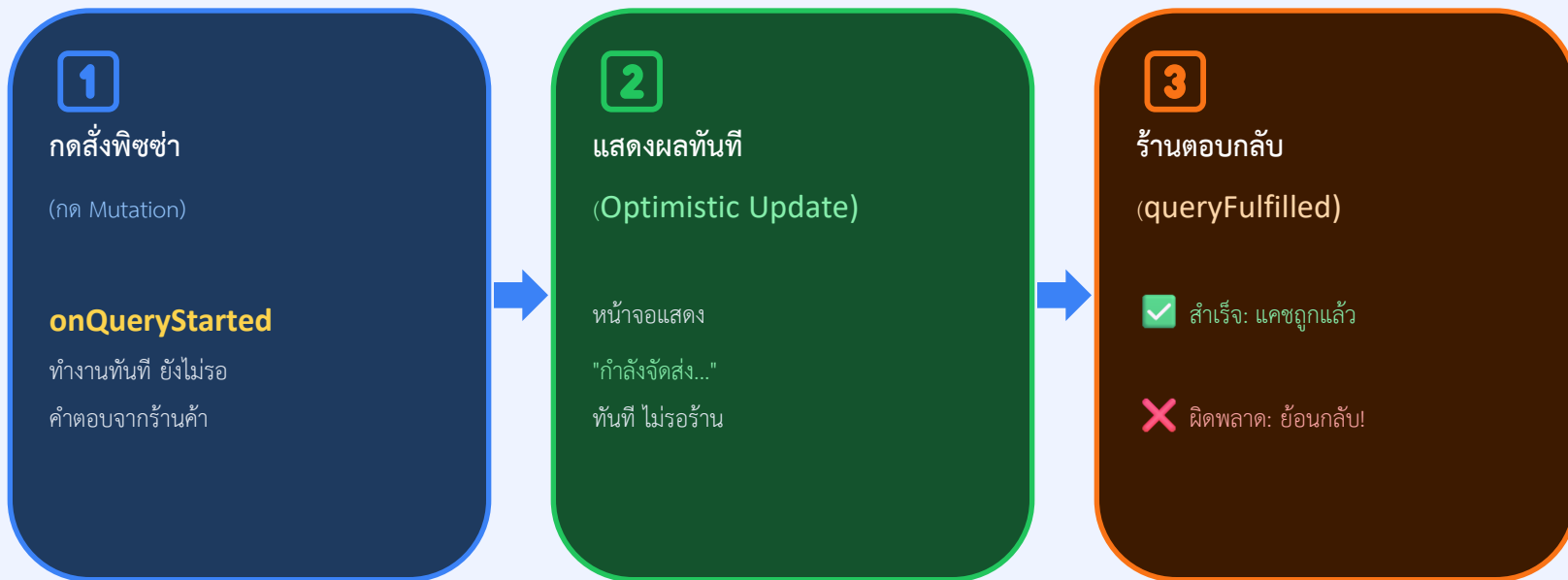
4 เกิดข้อผิดพลาด → ย้อนกลับอัตโนมัติ

UI ย้อนกลับ แจ้งผู้ใช้อย่างรวดเร็ว

onQueryStarted — เข้าใจง่ายๆ



เหมือนการสั่งพิซซ่าออนไลน์ — ระบบอัปเดตหน้าจอทันที ไม่รอให้ร้านยืนยัน



สรุปง่ายๆ: onQueryStarted คือ "ผู้ช่วยที่ตื่นตัว" — อัปเดต UI ก่อนเลย แล้วรอฟังเซิร์ฟเวอร์ ถ้าสำเร็จ: โอเค ถ้าผิดพลาด: ย้อนกลับทุกอย่าง

onQueryStarted — Mutation Lifecycle Hook

onQueryStarted ทำงานทันทีเมื่อ mutation ถูกเรียก — ก่อนเซิร์ฟเวอร์ตอบสนอง ใช้เพื่อ patch แคชแบบ optimistic

```
// onQueryStarted – inside a builder.mutation({ ... }) definition
async onQueryStarted(arg, {
  dispatch,          // store dispatch – use util.updateQueryData here
  getState,          // snapshot of current store state
  queryFulfilled,    // Promise<{ data, meta }> – resolves or rejects
  requestId,         // unique string ID for this mutation request
}) {
  try {
    const { data: serverResult } = await queryFulfilled;
    // Runs AFTER server responds successfully
    // Cache already patched optimistically → may not need to do anything
  } catch (err) {
    // Runs if server returns an error OR component unmounts early
    // Roll back any optimistic patches here
  }
},
```

- ใช้ได้ทั้ง builder.query() และ builder.mutation() — มีประโยชน์มากเมื่อใช้กับ mutations
- dispatch() ใน onQueryStarted คือ Redux dispatch เดียวกัน — ใช้ util.updateQueryData
- queryFulfilled reject เมื่อ: เซิร์ฟเวอร์ error, เครือข่ายล้มเหลว หรือ component ถูก unmount ก่อนได้รับผล

util.updateQueryData — เข้าใจง่ายๆ



เหมือนการแก้ไขสมุดเกรดนักเรียน — แก้ตรงในสมุด + มีสำเนาเพื่อย้อนกลับ



endpointName

"getStudents"

ชื่อหน้าสมุดที่จะแก้



queryArg

undefined

หมายเลขหน้า / ตัวกรอง

ที่ใช้เปิดสมุด



updateFn (draft)

ปากกาแก้ไขตรงสมุด

เขียนทับได้เลย

(Immer draft)



แก้ไขสมุดทันที

UI เห็นค่าใหม่ก่อนเซิร์ฟเวอร์ตอบ

(Optimistic Update)



patchResult.undo()

ย้อนกลับสมุดเหมือนเดิม

ถ้าเซิร์ฟเวอร์ตอบว่าผิดพลาด



สรุปง่ายๆ: updateQueryData = แก้สมุดเกรดโดยตรง | draft = ปากกาแก้ไข (Immer) | patchResult.undo() = ยางลบสำรองเพื่อย้อนกลับ

util.updateQueryData — Patch แคชทันที

updateQueryData patch ผลลัพธ์ในแคชทันที — ใช้ Immer จึง mutate draft โดยตรง คืนค่า patchResult พร้อม undo()

```
// studentsApi.util.updateQueryData(endpointName, queryArg, updateFn)
//
// endpointName → key in createApi endpoints (e.g. 'getStudents')
// queryArg      → exact arg used when the query was called
// updateFn      → receives an Immer draft; mutate it directly

const patchResult = dispatch(
  studentsApi.util.updateQueryData(
    'getStudents', // endpointName
    undefined,     // queryArg (useGetStudentsQuery() called with undefined)
    draft => {
      // draft is an Immer proxy of the cached Student[]
      const item = draft.find(s => s.id === updatedStudent.id);
      if (item) Object.assign(item, updatedStudent);
    }
  )
);

// patchResult.undo() — reverts the Immer patch atomically
// Safe to call even if queryFulfilled already resolved
patchResult.undo();
```


Optimistic updateStudent — ส่วนที่ 1/2

เพิ่ม onQueryStarted ใน mutation updateStudent เพื่อ patch ทั้ง list cache และ detail cache ทันที:

```
// src/features/students/studentsApi.js
updateStudent: builder.mutation({
  query: ({ id, ...patch }) => ({
    url: `students/${id}`,
    method: 'PUT',
    body: patch,
  }),
  async onQueryStarted({ id, ...patch }, { dispatch, queryFulfilled }) {
    // — Patch the list cache immediately —————
    const patchList = dispatch(
      studentsApi.util.updateQueryData(
        'getStudents', undefined,
        draft => {
          const item = draft.find(s => s.id === id);
          if (item) Object.assign(item, patch);
        }
      )
    );
    // — Patch the single-student detail cache —————
    const patchDetail = dispatch(
      studentsApi.util.updateQueryData(
        'getStudentById', id,
        draft => { Object.assign(draft, patch); }
      )
    );
  }
});
```

ต่อไปสไลด์ถัดไป → try/catch + rollback

Optimistic updateStudent — ส่วนที่ 2/2

รอมผลจากเซิร์ฟเวอร์ หากล้มเหลว ให้ undo ทั้งสอง cache patches แบบ atomic ด้วย patchResult.undo():

```
// — Wait for server — undo both patches on failure —  
try {  
  await queryFulfilled;  
  // Server confirmed → cache already shows correct data  
} catch {  
  // Server rejected → revert both patches atomically  
  patchList.undo();  
  patchDetail.undo();  
}  
},  
// Still invalidate to sync fine-grained { type, id } entries  
invalidatesTags: (result, error, student) => [{ type: "Student", id: student.id }, { type: "Student", id: "LIST" }],  
}),
```

วงจร optimistic แบบครบถ้วน:

1. เรียก mutation trigger

2. onQueryStarted ทำงาน —
patch แคช

3. ส่งคำขอเซิร์ฟเวอร์ใน
เบื้องหลัง

4. สำเร็จ → แคชถูกต้อง

4. ล้มเหลว → undo()

Polling & Refetching

รักษาข้อมูลให้ทันสมัยโดยอัตโนมัติ — `pollingInterval`, `refetchOnFocus`, `refetchOnReconnect` และ `manual triggers`

pollingInterval — Refetch อัตโนมัติตามเวลา

ตั้งค่า pollingInterval (มิลลิวินาที) ใน query hook เพื่อ refetch อัตโนมัติในช่วงเวลาที่กำหนดขณะ component ยังแสดงอยู่:

```
function StudentDashboard() {
  const {
    data: students = [],
    isLoading,
    isFetching, // true during background re-fetch (not first load)
  } = useGetStudentsQuery(undefined, {
    pollingInterval: 30_000, // 30_000 คือ 30,000 milliseconds = 30 วินาที (ตัว _ ใช้แทน comma เพื่ออ่านง่ายขึ้นใน JS) (0 = disabled)
  });

  if (isLoading) return <p>Loading...</p>;

  return (
    <div>
      {isFetching && (
        <span style={{ fontSize: 11, color: '#3A5BA0' }}>
          ⌚ Syncing...
        </span>
      )}
      <StudentTable students={students} />
    </div>
  );
}
```

// RTK Query deduplicates: two mounted components calling the same
// useGetStudentsQuery() share ONE subscription → ONE network request.
// Polling uses the shortest interval across all active subscriptions.

refetchOnFocus · refetchOnReconnect · skip

Flag ระดับ global ควบคุมเมื่อ RTK Query refetch อัตโนมัติ ตัวเลือกระดับ hook แต่ละตัวสามารถ override ค่า global ได้:

```
// — Global flags on createApi —————
export const studentsApi = createApi({
  reducerPath: 'studentsApi',
  baseQuery: fetchBaseQuery({ baseUrl: BASE }),
  refetchOnFocus: true, // re-fetch when window regains focus
  refetchOnReconnect: true, // re-fetch after network reconnect
  tagTypes: ['Student'],
  endpoints: builder => ({ ... }),
});
// Requires setupListeners in store.js
// import { setupListeners } from '@reduxjs/toolkit/query';
// setupListeners(store.dispatch);

// — Per-query overrides at hook level —————
const { data, refetch } = useGetStudentsQuery(undefined, {
  // Re-fetch every mount, not only when cache is stale:
  refetchOnMountOrArgChange: true,
  // Skip the query entirely (no request sent):
  skip: !isAuthenticated,
  // Override the global flag for this hook only:
  refetchOnFocus: false,
});
```

ฟังก์ชัน refetch ที่คืนมาจาก useXxxQuery() เรียก network request ทันที โดยข้าม cache TTL:

```
function StudentList() {
  const {
    data: students = [],
    isLoading,
    isError,
    error,
    refetch, // ← call to trigger a fresh network request
  } = useGetStudentsQuery();

  if (isLoading) return <p>Loading...</p>;
  if (isError) return (
    <div>
      <p>Error: {error?.data?.message ?? 'Unknown'}</p>
      <button onClick={refetch}>⏮ Retry</button>
    </div>
  );
  return (
    <>
      <button onClick={refetch}>⏮ Refresh</button>
      <table>...</table>
    </>
  );
}
```

SESSION 7

การแบ่งหน้า

รูปแบบแบ่งหน้าและ infinite scroll ด้วย `serializeQueryArgs`, `merge` และ `forceRefetch`

RTK Query รองรับทั้งสองแนวทาง เลือกตาม UI ของเรา: ปุ่มเลือกหน้า หรือ infinite scroll:

Offset / แบ่งตามหน้า	
Query param:	?page=2&limit=10
แคช:	หนึ่งรายการต่อหน้า — เปลี่ยนหน้าดึงข้อมูลใหม่
เหมาะกับ:	ตาราง Admin, รายงานที่ทราบจำนวนหน้าทั้งหมด
รูปแบบ RTK:	Query ปกติ — ไม่ต้องใช้ serializeQueryArgs
การใช้ Hook:	useGetStudentsPageQuery({ page, limit })
การนำทาง:	ปุ่ม ← ก่อนหน้า / ถัดไป →; ข้ามไปหน้า N
เชิร์ฟเวอร์ต้องการ:	จำนวนทั้งหมดเพื่อคำนวณจำนวนหน้า

Cursor / Infinite Scroll	
Query param:	?cursor=lastId&limit=10
แคช:	หนึ่งรายการ — หน้าใหม่ถูก merge โดยใช้ merge()
เหมาะกับ:	Social feeds, รายการไฟล์, ชุดข้อมูลขนาดใหญ่
รูปแบบ RTK:	serializeQueryArgs + merge + forceRefetch
การใช้ Hook:	useGetStudentsPageQuery({ page, limit })
การนำทาง:	ปุ่ม 'โหลดเพิ่ม' หรือ IntersectionObserver
เชิร์ฟเวอร์ต้องการ:	ไม่ต้องระบุจำนวนทั้งหมด — ใช้ cursor ถัดไป

serializeQueryArgs · merge · forceRefetch

มีสามตัวเลือกทำงานร่วมกันเพื่อ implement infinite scroll — ทุกหน้าสะสมอยู่ใน cache entry เดียว:

```
// Infinite-scroll endpoint in studentsApi
getStudentsPage: builder.query({
  query: ({ page, limit }) =>
    `students?page=${page}&limit=${limit}`,

  // Key the cache only by limit – ignore page number
  // All pages share ONE cache entry regardless of page value
  serializeQueryArgs: ({ queryArgs }) => queryArgs.limit,

  // Append new page data into the existing cached array
  merge: (currentCache, newItems) => {
    currentCache.push(...newItems);
  },

  // Force a real fetch even when the cache key hasn't changed
  forceRefetch: ({ currentArg, previousArg }) =>
    currentArg?.page !== previousArg?.page,

  providesTags: ['Student'],
}),
```

```
// Usage in component:
// const { data = [] } = useGetStudentsPageQuery({ page, limit: 10 });
// Incrementing page appends new items to data[]
```

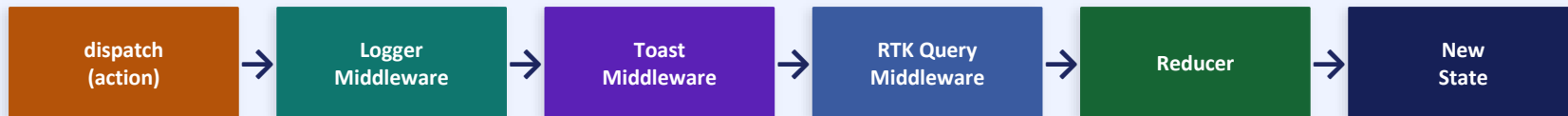
ใช้ local state ติดตามหน้าปัจจุบัน RTK Query จัดการดึงข้อมูล แคช และ merge รายการใหม่โดยอัตโนมัติ:

```
function PaginatedStudents() {
  const [page, setPage] = useState(1);
  const LIMIT = 10;
  const { data: students = [], isFetching, isLoading } =
    useGetStudentsPageQuery({ page, limit: LIMIT });
  return (
    <div>
      {isLoading && <p>Loading first page...</p>}
      {isFetching && <p className="syncing">⌚ Loading page {page}...</p>}
      <StudentTable students={students} />
      <div className="pagination-controls">
        <button onClick={() => setPage(p => Math.max(1,p-1))}>
          disabled={page===1|isFetching}< Prev</button>
        <span>Page {page}</span>
        <button onClick={() => setPage(p => p+1)}>
          disabled={isFetching}>Next →</button>
      </div>
    </div>
  );
}
```

Custom Middleware

ดักจับทุก action ที่ถูก dispatch เพื่อเพิ่ม logging, การแจ้งเตือน error และ analytics

Middleware อยู่ระหว่าง dispatch() และ reducer — ดักจับทุก action ตามลำดับ:



Signature:

```
const myMiddleware = storeAPI => next => action => {  
  // storeAPI.getState() - read current state  
  // storeAPI.dispatch() - dispatch another action  
  // next(action) - pass to the next middleware (or reducer)  
  return next(action); // MUST return this  
};
```

ใช้ middleware สำหรับความต้องการข้ามหลายส่วน:

- บันทึก action ทั้งหมดที่ถูก dispatch (ใช้คู่กับ Redux DevTools)
- แสดง UI toast เมื่อ RTK Query mutation เกิดข้อผิดพลาด
- ส่งเหตุการณ์ analytics ไปยัง GA / Mixpanel สำหรับ action type ที่กำหนด
- รายงาน crash — บันทึก action + state snapshot เมื่อเกิดข้อผิดพลาด

บันทึกทุก action ที่ถูก dispatch พร้อม state ก่อนและหลัง — เฉพาะ development mode เพื่อไม่ส่งผลต่อ production:

```
// src/app/middleware/logger.js
// Middleware signature: storeAPI => next => action => returnValue
const loggerMiddleware = storeAPI => next => action => {
  if (process.env.NODE_ENV !== 'development') return next(action);

  console.group(
    `%c Action: ${action.type}`,
    'color:#3A5BA0; font-weight:bold'
  );
  console.log('%c prev state', 'color:#9E9E9E', storeAPI.getState());
  console.log('%c action', 'color:#03A9F4', action);
  const result = next(action);
  console.log('%c next state', 'color:#4CAF50', storeAPI.getState());
  console.groupEnd();
  return result;
};

export default loggerMiddleware;
```

ใน browser console จะเห็นสำหรับทุก RTK Query operation:

► Action: studentsApi/executeQuery/fulfilled [prev state] [action] [next state]

ใช้ `isRejectedWithValue` จาก Redux Toolkit เพื่อตรวจจับ mutation ที่ล้มเหลวและแสดง toast notification:

```
// src/app/middleware/toast.js
// isRejectedWithValue – true when mutation.rejected carries a payload
import { isRejectedWithValue } from '@reduxjs/toolkit';

const toastMiddleware = _storeAPI => next => action => {
  if (isRejectedWithValue(action)) {
    // action.payload is what transformErrorResponse returned
    const status = action.payload?.status ?? 'ERR';
    const message = action.payload?.data?.message ?? 'Request failed';

    // Swap console.error for your toast library, e.g.:
    // import toast from 'react-hot-toast';
    // toast.error(`[${status}] ${message}`);
    console.error(`API [${status}] ${message}`);
  }
  return next(action);
};

export default toastMiddleware;
```

เชื่อมต่อ middleware ทั้งหมดใน configureStore ลำดับสำคัญ — middleware ทำงานจากซ้ายไปขวาเมื่อ dispatch:

```
// src/app/store.js - full middleware chain
import { configureStore } from '@reduxjs/toolkit';
import { studentsApi } from '../features/students/studentsApi';
import { setupListeners } from '@reduxjs/toolkit/query';
import loggerMiddleware from './middleware/logger';
import toastMiddleware from './middleware/toast';

export const store = configureStore({
  reducer: {
    [studentsApi.reducerPath]: studentsApi.reducer,
    // other slices ...
  },
  middleware: getDefaultMiddleware =>
    getDefaultMiddleware()
      .concat(studentsApi.middleware) // caching, invalidation, polling
      .concat(loggerMiddleware)       // dev console logging
      .concat(toastMiddleware),       // error notifications
});

// Enables refetchOnFocus + refetchOnReconnect listeners:
setupListeners(store.dispatch);

export default store;
```

SESSION 7

Lab 7

เพิ่ม optimistic updates, เปิด live polling และเชื่อมต่อ logger middleware ใน AcadeMate

แบบฝึกหัดสามข้อต่อยอดจากการตั้งค่า RTK Query ใน Session 6:

ขั้นตอนที่ 1

Optimistic updateStudent

เพิ่ม onQueryStarted ใน mutation
updateStudent Patch ทั้ง list และ detail
cache ทันที Undo เมื่อล้มเหลว

ขั้นตอนที่ 2

Polling + refetchOnFocus

เพิ่ม pollingInterval และ
refetchOnFocus/Reconnect ใน
StudentTable แสดง badge ขณะ isFetching

ขั้นตอนที่ 3

Logger Middleware

สร้าง middleware/logger.js และเพิ่มใน store
ตรวจสอบว่า action log ปรากฏใน browser
DevTools console

ไฟล์ที่ต้องแก้ไข / สร้าง

`src/features/students/studentsApi.js`

เพิ่ม onQueryStarted + try/catch ใน updateStudent mutation

`src/features/students/StudentTable.jsx`

เพิ่มตัวเลือก pollingInterval, refetchOnFocus, refetchOnReconnect

`src/app/middleware/logger.js`

สร้าง — action logger สำหรับ dev mode

`src/app/store.js`

เพิ่ม loggerMiddleware ใน chain; ยืนยันว่า setupListeners มีอยู่

Lab ขั้นตอนที่ 1 — Optimistic updateStudent (ส่วนที่ 1/2)

เปิด studentsApi.js และเพิ่ม onQueryStarted ใน mutation updateStudent Patch ทั้งสองแคชก่อนรอเซิร์ฟเวอร์:

```
// src/features/students/studentsApi.js
updateStudent: builder.mutation({
  query: ({ id, ...patch }) => ({
    url: `students/${id}`,
    method: 'PUT',
    body: patch,
  }),
  async onQueryStarted({ id, ...patch }, { dispatch, queryFulfilled }) {
    const patchList = dispatch(
      studentsApi.util.updateQueryData(
        'getStudents', undefined,
        draft => {
          const item = draft.find(s => s.id === id);
          if (item) Object.assign(item, patch);
        }
      )
    );
    const patchDetail = dispatch(
      studentsApi.util.updateQueryData(
        'getStudentById', id,
        draft => { Object.assign(draft, patch); }
      )
    );
  }
});
```

ต่อไปสไลด์ถัดไป → try/catch rollback

เติม body ของ onQueryStarted ด้วย try/catch หากเซิร์ฟเวอร์ปฏิเสธ เรียก undo() ทั้งสองผลลัพธ์เพื่อย้อนแซะ:

```
try {
  await queryFulfilled;
  // Server confirmed → cache is already correct, nothing to do
} catch {
  // Server rejected → revert both patches atomically
  patchList.undo();
  patchDetail.undo();
}
},
invalidatesTags: (result, error, { id }) =>
  [{ type: 'Student', id }],
}),
```

ทดสอบ:

ใน AcadeMate แก่ชื่อนักศึกษาและสังเกตตารางอัปเดตทันทีก่อนเซิร์ฟเวอร์ตอบสนอง จากนั้นพิมพ์ URL ผิดเพื่อบังคับให้เกิด 404 — ชื่อควรย้อนกลับอัตโนมัติ

คำแนะนำ: ใช้แท็บ Network ใน DevTools → คลิกขวา → Block request URL เพื่อจำลองความล้มเหลวอย่างสะอาด

Lab ขั้นตอนที่ 2 — Polling & refetchOnFocus ใน StudentTable

อัปเดต StudentTable.jsx ให้ poll ทุก 30 วินาที และ refetch เมื่อแท็บเบราว์เซอร์ได้รับ focus หรือเครือข่ายเชื่อมต่อใหม่:

```
// src/features/students/StudentTable.jsx
import { useGetStudentsQuery } from './studentsApi';
function StudentTable() {
  const {
    data: students = [],
    isLoading,
    isFetching,
    refetch,
  } = useGetStudentsQuery(undefined, {
    pollingInterval: 30_000, // re-fetch every 30 s
    refetchOnFocus: true,    // re-fetch on window focus
    refetchOnReconnect: true, // re-fetch after reconnect
    refetchOnMountOrArgChange: true, // always fresh on mount
  });
  if (isLoading) return <p>Loading students...</p>;
  return (
    <div>
      {isFetching && <span className="badge">🔄 Syncing...</span>}
      <button onClick={refetch}>🔄 Refresh</button>
      <table>{/* ... student rows ... */}</table>
    </div>
  );
}
```

สร้างไฟล์ logger middleware และเพิ่มใน middleware chain ของ store ตรวจสอบ action log ใน browser console:

```
// src/app/middleware/logger.js
const loggerMiddleware = storeAPI => next => action => {
  if (process.env.NODE_ENV !== 'development') return next(action);
  console.group(`Action: ${action.type}`);
  console.log('prev:', storeAPI.getState());
  console.log('action:', action);
  const result = next(action);
  console.log('next:', storeAPI.getState());
  console.groupEnd();
  return result;
};
export default loggerMiddleware;

// src/app/store.js - add logger to the chain
middleware: getDefaultMiddleware =>
  getDefaultMiddleware()
    .concat(studentsApi.middleware)
    .concat(loggerMiddleware),

// Verify in browser DevTools → Console:
// You should see grouped logs for every RTK Query operation
// e.g. "Action: studentsApi/executeQuery/fulfilled"
// with prev state → action payload → next state
```

ทดสอบความรู้

คำถาม:

แอปของคุณเรียก useGetStudentsQuery() ใน StudentTable และ mount hook เดียวกันใน Header badge ด้วย คุณตั้ง pollingInterval: 5000 เปิดแอปใน 2 แท็บเบราว์เซอร์ มี API request กี่ครั้งทุก 5 วินาที?

```
// A: 4 requests per 5 s (one per component per tab)
// B: 2 requests per 5 s (one per tab - deduplicated within each store)
// C: 1 request per 5 s (RTK Query deduplicates globally)
// D: 0 requests per 5 s (pollingInterval requires refetchOnFocus: true)
```

คำตอบ:

B — 2 requests ทุก 5 วินาที ภายใน Redux store เดียว RTK Query รวม subscription ซ้ำ: ทั้งสอง component ใช้ subscription เดียว → คำขอเดียว แต่แต่ละแท็บเบราว์เซอร์มี Redux store แยกกัน จึง poll อิสระ → รวม 2 ครั้ง

ข้อสังเกตสำคัญ:

การรวมซ้ำใช้ได้ภายใน store เดียว; แท็บแยก = store แยก = timer polling แยกกัน

แนวคิดหลักที่ครอบคลุมใน Session 7:

onQueryStarted

Async lifecycle hook — ทำงานทันทีเมื่อ mutation ถูก dispatch ก่อนเซิร์ฟเวอร์ตอบสนอง

pollingInterval

Refetch อัตโนมัติทุก N ms ขณะ mount รวม subscription ต่อ store — หลาย component ใช้คำขอเดียว

Custom Middleware

storeAPI → next → action ใช้สำหรับ logging, toast errors และ analytics เชื่อมด้วย .concat()

util.updateQueryData

Cache patch แบบ Immer — mutate draft โดยตรง คืนค่า patchResult.undo() สำหรับย้อนกลับ

serializeQueryArgs + merge

รูปแบบ Infinite scroll — แคชทุกหน้าใน key เดียว หน้าใหม่เพิ่มผ่าน merge()

setupListeners

จำเป็นสำหรับ refetchOnFocus + refetchOnReconnect เรียกครั้งเดียวด้วย store.dispatch

ตัวอย่างล่วงหน้า — Session 8: การทดสอบและ Deploy

Session 8 เสร็จสิ้นโปรเจกต์ AcadeMate ด้วยชุดการทดสอบครบถ้วนและ production build:

Vitest + RTL

Unit test reducers, selectors และ React components ด้วย Testing Library

MSW (Mock Service Worker)

Mock การตอบสนอง RTK Query API ระดับ network สำหรับการทดสอบที่สมจริง

การทดสอบ Async Flows

ทดสอบ optimistic updates, การยกเลิก polling และ error rollback แบบ end-to-end

Production Build

การ optimize Vite build, code splitting และ deploy AcadeMate ไปยัง Vercel

แหล่งอ้างอิงและอ่านเพิ่มเติม — Session 7

RTK Query — Optimistic Updates

redux-toolkit.js.org/rtk-query/usage/optimistic-updates

สไลด์ 5-8

RTK Query — Polling

redux-toolkit.js.org/rtk-query/usage/polling

สไลด์ 10-12

RTK Query — การแบ่งหน้า

redux-toolkit.js.org/rtk-query/usage/pagination

สไลด์ 15-16

Redux — การเขียน Middleware

redux.js.org/understanding/history-and-design/middleware

สไลด์ 18-21